

zod × TestPilot: Survival of LLM-Generated Tests Across Six Releases

Generated unit tests for zod 4.0.5 with a TestPilot-family tool, then ran them unchanged against the six later minor releases, side by side with the developers' own test suite frozen at 4.0.5. Date: 2026-09-20. Full details, per-case records and the deviation register: [testpilot-zod-survival-report-full.md](#).

1. Question

When a set of LLM-generated tests passes at time t, how much of it still passes at later releases, and how does that compare with the developers' own tests from the same time t? Both corpora are run on exactly the same code at each release and neither is edited.

2. Cases

- Subject: zod (TypeScript schema validation library). Anchor t = v4.0.5 (2025-07-10). Later points: v4.1.0, v4.2.0, v4.3.0, v4.4.0, v4.5.0, v4.6.0 (2025-08-23 to 2026-09-09). Between the anchor and v4.6.0 the library package received 421 commits (706 in the whole repository) and 38 patch releases, which are not used.
- Sampled functions: 60 of the 124 user-callable top-level functions of the main entry point (`z.<name>`, `z.coerce.*`, `z.iso.*`), stratified by role (36 schema factories, 17 checks and refinements, 7 parsing, coercion, format and configuration functions), seed recorded. `z.string` and `z.object` were not drawn.
- Generation: testpilot2 (the chat-model follow-up of TestPilot), model `gpt-oss-120b` via OpenRouter, temperature 0, one Mocha test per prompt, prompts built from the function signature, its body, up to three documentation snippets, and one retry with the error message. 360 prompts, 360 tests, 139 passing at t. These 139 are the LLM corpus S.
- Developer corpus: the 81 test files (888 cases) of zod's own suite at v4.0.5, all passing at t.

3. Protocol

For each release the source tree was exported and built, the generated tests were run one file at a time under Mocha with a 5-second timeout against the built package, and the developer suite from v4.0.5 was copied over the release's own tests and run under the release's Vitest. v4.0.5 itself was run through the same pipeline first as a control (139/139 and 888/888). Failures were categorised automatically by message: snapshot mismatch, assertion error, other. Release-level rather than commit-level sampling was chosen because the generated tests import the built package (`require('zod')`), so each point needs a build, and releases are the unit at which users upgrade.

4. Results

Release	Date	Generated tests passing / 139	Developer cases passing / 888	Developer failures: snapshot / assertion / other
v4.1.0	2025-08-23	139 (100.0%)	871 (98.1%)	12 / 5 / 0
v4.2.0	2025-12-14	139 (100.0%)	868 (97.8%)	13 / 6 / 0
v4.3.0	2025-12-30	139 (100.0%)	864 (97.3%)	14 / 7 / 2
v4.4.0	2026-04-29	139 (100.0%)	853 (96.1%)	20 / 10 / 4
v4.5.0	2026-08-28	138 (99.3%)	838 (94.4%)	34 / 12 / 3
v4.6.0	2026-09-09	138 (99.3%)	834 (93.9%)	34 / 16 / 3

One developer case is reported as skipped from v4.2.0 on (a runner version difference, not a code change); it stays in the denominator.

The only generated-test failure. One test, sampled for `z.parse` but exercising `z.iso.datetime()`, asserts that "2020-01-01T06:15Z" (no seconds) is accepted by the default `datetime` schema. Commit `036b39f4` (2026-08-24), titled `fix(v4)!: require seconds once a datetime carries a Z or an offset, made that input invalid`; the developers marked it as a breaking change and wrote migration notes. The same commit breaks the developer case "datetime parsing with local and offset". So the one generated-test break is a documented public-contract change that the developer suite also caught.

Developer failures. 53 cases fail at v4.6.0, in 23 files. 34 are snapshot mismatches (`toMatchInlineSnapshot` against stored JSON Schema output, regex text, or issue arrays), 15 of them in `to-json-schema.test.ts`. The 19 non-snapshot failures are behaviour changes: `preprocess` now records one non-fatal issue instead of two; a malformed UUID that was rejected is now accepted; `pick/omit` stopped throwing on unknown keys; template-literal failure cases no longer throw; `optional` and `lazy` report optionality as 'defaulted' instead of 'optional'; `catch` with enums throws; an internal locale helper `parsedType` was removed; and at v4.6.0 five computed properties in the mini API (`size`, `int32`, `int64`, `min/max`, `multipleOf`) became undefined.

Non-monotonic cases. Three developer cases broke and later passed again: two `catch` enum cases fail at v4.4.0 only, and the `pick/omit` unknown-key case fails from v4.1.0 through v4.5.0 and passes at v4.6.0. Even at release granularity, breakage is not always permanent.

Generation at t, for context. Test pass rate at t was 38.6% (139/360). Snippet-bearing prompts produced the most passing tests (33

of 58), the signature-only base prompts the fewest (23 of 60), in line with the paper. Most of the 221 failures are assertion errors in which the model assumed zod v3 (method names, error shapes, message wording) although the code was v4.

5. Observations

1. Survival rate measures maintenance burden, not test quality, and by itself cannot separate two kinds of break (the public contract really changed, or the test asserted something that is not a contract: message text, snapshot, internal structure) or two kinds of survival (the contract did not change, or it changed and the test did not notice). The separation needs the two corpora paired per function: both broke (a real change, detected), developer only (the generated test missed it), generated only (brittle), neither (no information). The one generated break is in the "both" cell. `preprocess` and `uuid` are sampled functions whose developer cases broke while every generated test for them survived; these are candidates for the "missed" cell. Automatic file-name pairing covers only 15 of the 54 functions with passing tests; the other 39 have their developer cases inside type-level files (`string.test.ts` alone holds 16 of the sampled string-format functions) and in one file that has one case per API. Pairing therefore has to be done at case level with a per-function judgement; this is the design decision to settle next.
2. Why the generated tests almost never broke. The 139 survivors are short (median 26 lines) and assert the most stable layer of the API: valid input parses to the same value (`strictEqual`, 99 tests), invalid input throws (`throws`, 88 tests), valid input does not throw (54 tests). Only 30 tests match an error message with a regular expression and 14 read the issue structure. In addition, the pass gate at `t` had already removed the brittle tests: 156 of the 221 failures at `t` were assertion errors on v3-style messages and error shapes. The survival rate is thus largely a consequence of how `S` was defined.
3. Two kinds of staleness. In the summer experiments tests went stale because the code changed after they were written. Here most generated tests were stale at birth: the model wrote zod v3 tests for zod v4 code (61% failed at `t`). The survival stage measures only the tests that were valid at `t`.
4. Developer tests also assert non-contract facts. Two thirds of the developer failures (34 of 53) are inline-snapshot mismatches. Snapshot testing is not specific to JavaScript (ApprovalTests in Java, for example), so the same category of break should be expected in the Java repositories.
5. Release granularity. The unit of observation follows the layer the tests attach to. The summer tests imported source files by relative path, so commits were a natural unit and no build was needed. These tests import the published package, so releases are the consistent unit and every point needs a build. What is lost is the identity of the causing commit and any transient break between releases; the three non-monotonic cases show such breaks exist. Both losses can be recovered selectively: bisect only the broken cases (the widest interval is 145 commits, about 8 builds), or extend to the 38 patch tags (44 points at about 80 s each).

6. Coverage

Coverage was not measured in this track. The reason is the execution setup: zod's package is ES-module only, so the tool's CommonJS test files cannot load it directly, and a one-line wrapper package is used instead. The tool's coverage instrumentation (`nyc`) skips `node_modules`, so it measures only the wrapper file and nothing of zod. This was confirmed on a probe before generation and recorded as a limitation; the survival runs then dropped `nyc` entirely.

Coverage would serve two purposes here. First, the paper's own metric: how much of each sampled function the generated tests execute, compared with the developer suite. Second, and specific to survival: for functions in the "developer broke, generated survived" cell, coverage tells whether the generated tests executed the changed path at all or simply never reached it. Three ways to obtain it, in order of cost: run the same Mocha runner under `c8` (V8's built-in coverage, no instrumentation, on the built `.cjs` files with source maps back to the TypeScript source); reconfigure `nyc` to include the symlinked package; or move the generated tests to a Vitest runner against the TypeScript source as in the week-1 pilot, which changes the runner and would require re-confirming pass/fail. The first is the cheapest and does not touch the execution structure.

7. Limitations

- Generation prompts on the built JavaScript, not on the TypeScript source, and the build strips doc comments (0 of 569 discovered functions had one), so the paper's doc-comment prompt variant could not occur.
- One completion per prompt (tool behaviour), `max_tokens` 4000 and three snippets per function differ from the paper's settings; `testpilot2`'s prompts differ from the paper's `TestPilot`, so figures are not compared with the paper's numbers.
- Coverage not measured (6); OpenRouter provider not pinned.
- The definition of `S` (tests passing at `t`) removes brittle tests before survival is measured (observation 2).
- Release-level sampling loses causing commits and transient breaks (observation 5).
- v4.6.0 changed the monorepo's package manager; our build bypassed its wrapper and resolved dependencies afresh, so its developer-test environment is not the release's own. The generated tests are unaffected (they need only the built entry file).
- Failure categories are automatic; the per-case contract / non-contract classification and the case-level pairing are not done yet.
- 60 of 124 functions sampled; `z.string` and `z.object` absent; the `mini` entry point not sampled.

8. Next steps (for discussion)

Case-level pairing of the two corpora and the 2x2 readout; manual contract / non-contract classification of the 53 developer failures; coverage via `c8` on the existing runner; bisection of the broken cases or extension to patch tags; the remaining 64 functions and the `mini` entry point.

Repository: `Hamjoon/zod`, branch `experiment/2026-09-week3-testpilot-zod`, directory `experiments/testpilot-2026-09/`. Full report:

